

Lab 5

OOP

Lab Manual: Implementation of the Concept of Inheritance in Java

1. Objective

The objective of this lab is to:

1. Understand the concept of **Inheritance** in Java.
2. Learn about **types of inheritance** .
3. Implement **method overriding** and use the **super keyword** in Java.
4. Work on **real-world scenario-based problems** to apply inheritance.

2. Introduction to Inheritance

What is Inheritance?

Definition:

Inheritance is a fundamental concept in **Object-Oriented Programming (OOP)** that allows one class (child class) to acquire the properties and behavior of another class (parent class).

Why is Inheritance Important?

1. **Code Reusability:** Avoids writing the same code multiple times.
2. **Better Organization:** Helps in structuring code efficiently.
3. **Extensibility:** New features can be added easily without modifying existing code.
4. **Readability & Maintenance:** Inherited code makes the program more readable.

3. How Inheritance Works in Java?

In Java, inheritance is achieved using the **extends** keyword.

1. The **Parent Class (Super Class)**: The class that is being inherited.
2. The **Child Class (Sub Class)**: The class that inherits from another class.
3. The **extends keyword**: Used to establish an inheritance relationship.

Syntax of Inheritance in Java

```
class Parent {  
    // Parent class properties and methods  
}  
  
class Child extends Parent {  
    // Child class properties and methods  
}
```

4. Types of Inheritance in Java

Type	Description	Example
1. Single Inheritance	One child class inherits from one parent class.	class Car extends Vehicle {}
2. Multilevel Inheritance	A class inherits from another child class.	class SportsCar extends Car {}
3. Hierarchical Inheritance	Multiple child classes inherit from one parent class.	class Bike extends Vehicle {}

5. Key Concepts in Inheritance

5.1 Parent Class (Superclass)

A class whose properties and methods are inherited by another class.

5.2 Child Class (Subclass)

A class that inherits from another class and can add or modify functionalities.

5.3 Super Keyword

Used to call parent class methods or constructors inside a child class.

5.4 Method Overriding

When a child class provides a **new implementation** of a method already defined in the parent class.

5.5 This keyword

The **this** keyword refers to **the current object** in a class

6. What is a Constructor?

A **constructor** is a special method in a class that **runs automatically** when an object is created.

Its **name is always the same as the class name**, and it **initializes the object's values**

Types of Constructors

1. **Default Constructor** – Created automatically if you don't write one.
2. **Parameterized Constructor** – Takes values when an object is created.
3. **Copy Constructor** – Copies values from one object to another.

Simple Example of a Constructor

```
class Student {  
    String name;  
    int age;  
    // Constructor jab object create hota hai tab automatic chalta hai  
    Student(String studentName, int studentAge) {  
        name = studentName; // Name ko initialize kra lia  
        age = studentAge; // Age ko initialize kra lia  
    }  
    // Student ki details print karne wali method  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}  
  
public class ConstructorExample {  
    public static void main(String[] args) {  
        Student s1 = new Student("Hassan", 23);  
        s1.display();  
    }  
}
```

7. What is the this keyword?

The **this** keyword refers to **the current object** in a class.

It helps when the parameter names and class attributes have the same names.

Example of this keyword

```
class Car {
    String model;
    // Constructor jo model ka naam set karega
    Car(String model) {
        // this keyword use kar rahe hain taake class variable aur parameter dono ko differentiate
        // karein
        this.model = model;
    }
    void display() {
        System.out.println("Car Model: " + model);
    }
}

public class ThisKeywordExample {
    public static void main(String[] args) {
        // Car ka ek object create kar rahe hain aur usko Toyota value de rahe hain
        Car c1 = new Car("Toyota");
        c1.display();
    }
}
```

8. Scenario-Based Examples

Scenario : Employee Management System

Problem Statement:

Create a class **Employee** with properties name and salary. Create a subclass

Manager that has an additional property **bonus**.

```
class UniversityEmployee {
    String name;
    double salary;
    // Constructor jo name aur salary ko initialize karta hai.
    UniversityEmployee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Salary: $" + salary);
    }
}
```

```
}  
}  
class DepartmentManager extends UniversityEmployee {  
    double bonus;  
    // Constructor jo name, salary aur bonus ko initialize karega.  
    DepartmentManager(String name, double salary, double bonus) {  
        // Department manager ka ek additional attribute "bonus" hoga.  
        super(name, salary); // Super ka use parent class ka constructor call karne ke  
        // liye.  
        this.bonus = bonus;  
    }  
    void display() {  
        super.display();  
        System.out.println("Bonus: $" + bonus);  
    }  
}  
public class UniversityExample {  
    public static void main(String[] args) {  
  
        DepartmentManager dm1 = new DepartmentManager("Dr. Hassan", 70000,  
15000);  
        dm1.display();  
    }  
}
```